

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Análisis y Diseño de Algoritmos

Práctica:

Algoritmos básicos.

Fecha: 04/09/25

Domínguez Jiménez Ana Andrea

a)

```c
//Ejercicio 1. Funcion Multiply(x,y)
#include<stdio.h>
#include<stdlib.h>
int multiply(int x, int y){
    if (y == 0) {
        return 0;
    }

    int z = multiply(x, y / 2);

    if (y % 2 == 0) {
        return 2*z;
    } else {
        return x+2*z;
    }
}

int main(){
    int  x, y, resultado;

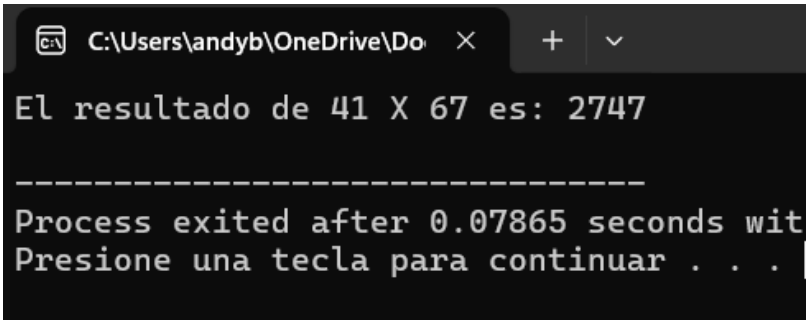

    x = rand() % 100;
    y= rand() % 10000;

    if(y < 0){
        printf("\nEl valor de y debe ser mayor o igual a cero");
    }

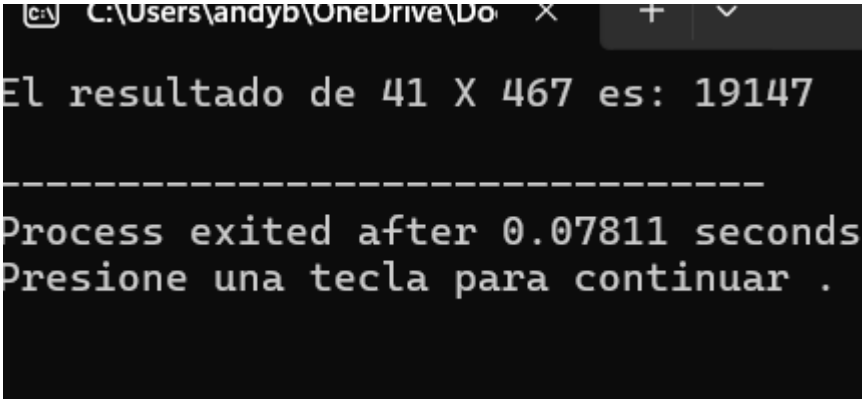
    resultado = multiply(x, y);
    printf("El resultado de %d X %d es: %d\n", x, y, resultado);


    return 0;
}
```

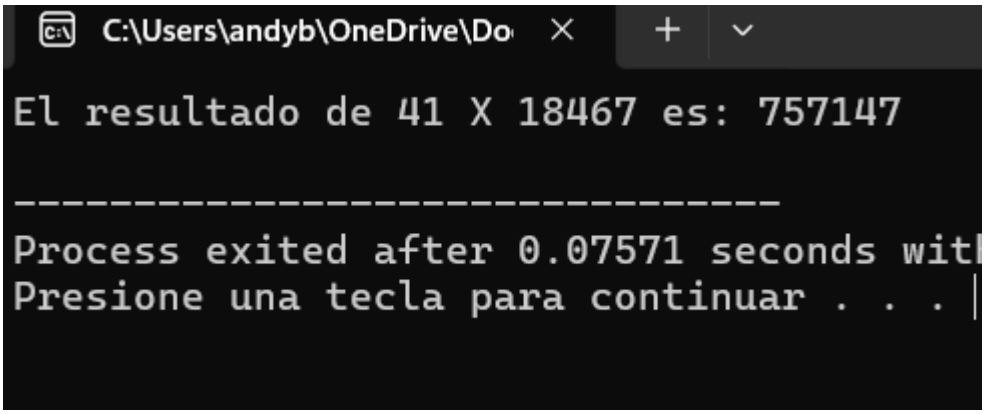
Ejemplo con 2 dígitos:



Ejemplo con 3 dígitos:



Ejemplo con 5 dígitos:



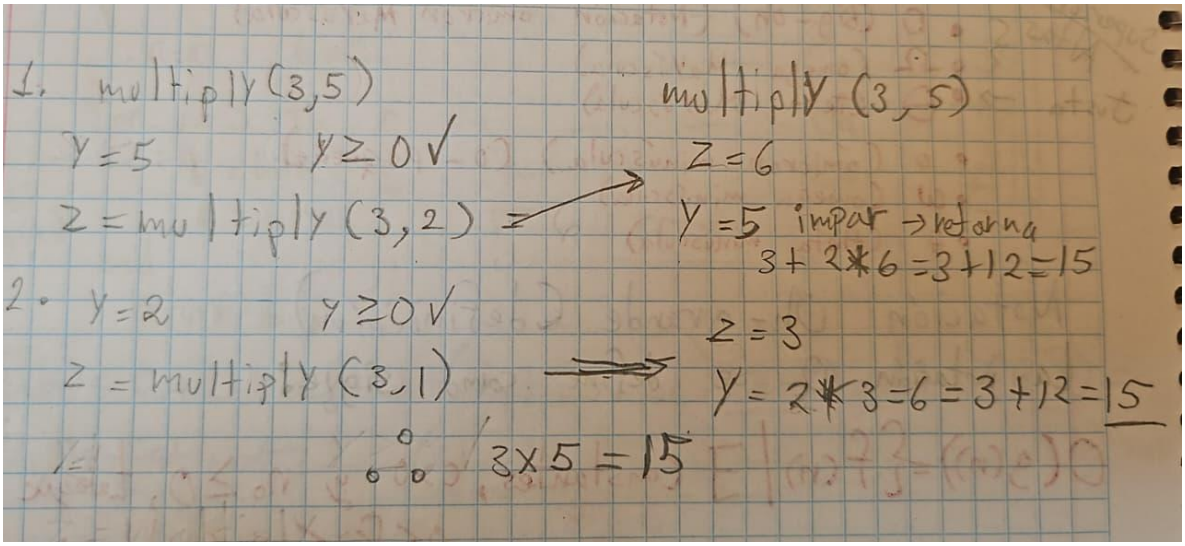
Ejemplo con 10 dígitos:

```
Ingrese el valor x:78
Ingrese el valor y: 7896541235

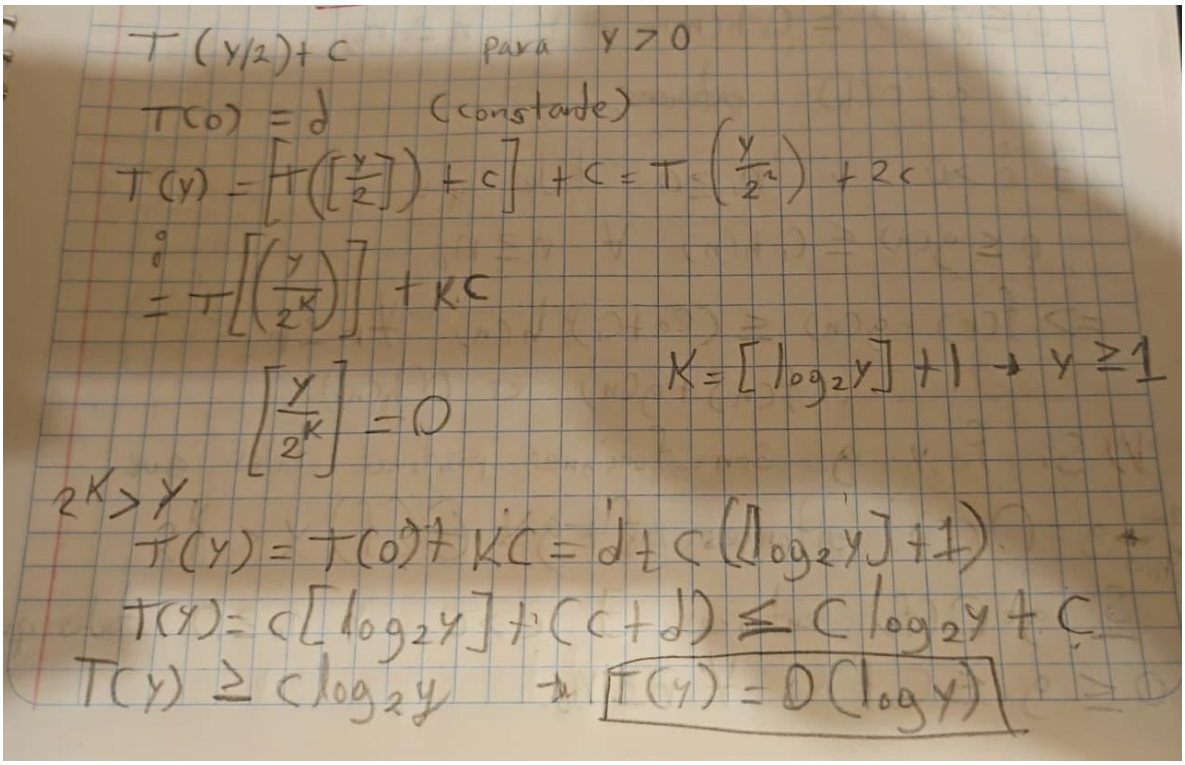
El valor de y debe ser mayor o igual a ceroEl resultado de 78 X -693393357 es: -1749893002

--------------------------------
Process exited after 4.908 seconds with return value 0
Presione una tecla para continuar . . .
```

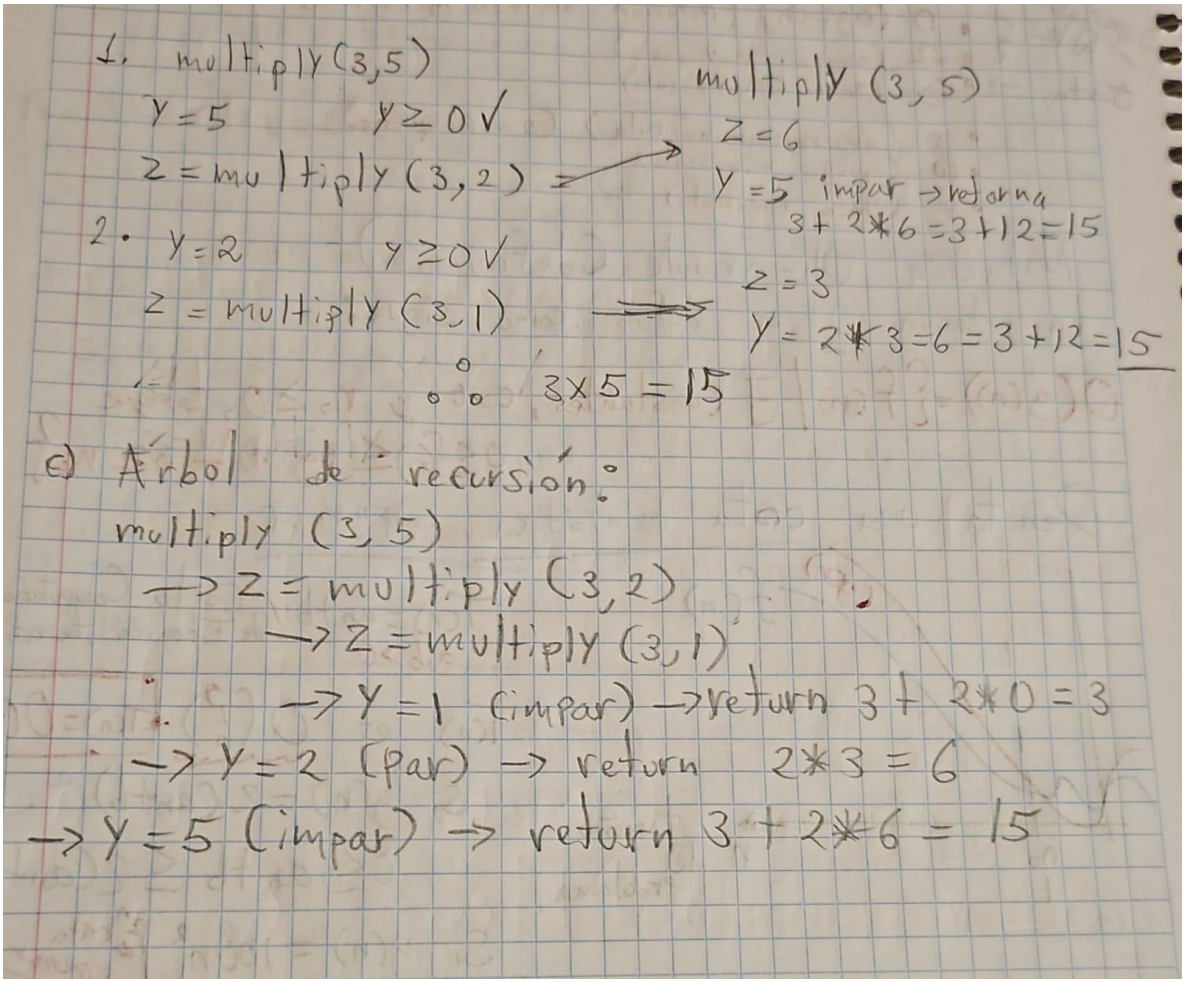
Ejemplo de que las operaciones son correctas:



b)


T(y/2)+c para y>0
T(0) = d (constante)
T(y) = [T([y/2]) + c] + c = T(y/2^2) + 2c
= T[y/2^k] + kc
[y/2^k] = 0
K = [log2 y] + 1 → y ≥ 1
2^K > y
T(y) = T(0) + kc = d + c([log2 y] + 1)
T(y) = c[log2 y] + (c + d) ≤ c log2 y + c
T(y) ≥ c log2 y → T(y) = O(log y)

c)


1. multiply(3,5)
y=5 y≥0 ✓
z=multiply(3,2)
multiply (3, 5)
z=6
y=5 impar → retorna
3+2*6=3+12=15
2. y=2 y≥0 ✓
z=multiply(3,1)
z=3
y=2*3=6=3+12=15
∴ 3x5=15
c) Árbol de recursión:
multiply (3, 5)
→ z=multiply(3,2)
→ z=multiply(3,1)
→ y=1 (impar) → return 3+2*0=3
→ y=2 (par) → return 2*3=6
→ y=5 (impar) → return 3+2*6=15

Ejercicio_2.cpp

```cpp
//Ejercicio 2.
#include <stdio.h>

int main() {
    int A[100], B[100][100];
    int n, i, j, k;


    printf("¿Cuántos elementos tiene el arreglo A? ");
    scanf("%d", &n);

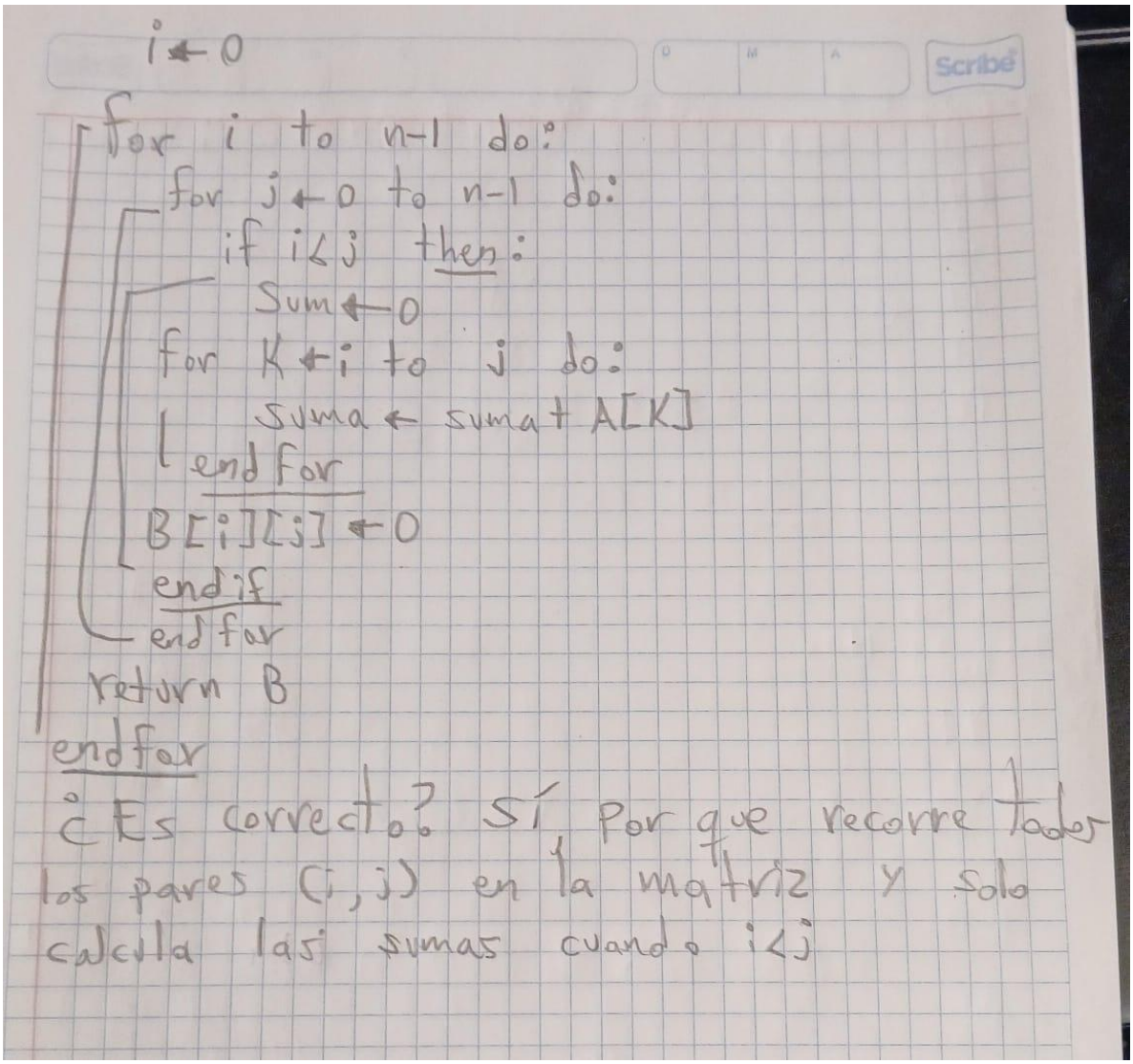
    printf("Ingresa los %d elementos de A:\n", n);
    for (i = 0; i < n; i++) {
        printf("A[%d]: ", i);
        scanf("%d", &A[i]);
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (i < j) {
                int suma = 0;
                for (k = i; k <= j; k++) {
                    suma += A[k];
                }
                B[i][j] = suma;
            } else {
                B[i][j] = 0;
            }
        }
    }

    printf("\nMatriz B:\n");
    for (i = 0; i < n; i++) {
```

```
        for (j = 0; j < n; j++) {
            printf("%4d ", B[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```



Ejercicio_3.cpp

```cpp
// Ejercicio 3: Evaluación de polinomios
#include <stdio.h>
#include <math.h>


int Clasico(int poli[], int grado, int x) {
    int resultado = 0;
    for (int i = 0; i <= grado; i++) {
        resultado += poli[i] * (int)pow(x, i);
    }
    return resultado;
}


int Horner(int poli[], int grado, int x) {
    int resultado = poli[grado];
    for (int i = grado - 1; i >= 0; i--) {
        resultado = resultado * x + poli[i];
    }
    return resultado;
}

int main() {
    int poli[100];
    int grado, x;

    printf("Ingresa el grado del polinomio: ");
    scanf("%d", &grado);

    if (grado < 100) {
        for (int i = 0; i <= grado; i++) {
            printf("Ingresa el coeficiente a%d: ", i);
            scanf("%d", &poli[i]);
```

```
34          }
35
36          printf("Ingresa el valor de x: ");
37          scanf("%d", &x);
38
39          int resClasico = Clasico(poli, grado, x);
40          int resHorner =
41          Horner(poli, grado, x);
42
43          printf("\nResultado usando método clasico: %d\n", resClasico);
44          printf("Resultado usando método de Horner: %d\n", resHorner);
45
46      } else {
47          printf("El grado debe ser menor que 100\n");
48      }
49
50      return 0;
51  }
52
```

Clásico

	num operaciones
res = 0	1
for i = 0 ... n:	n+1
pot = 1	1
for j = 1 ... i:	n
pot = pot * X	n
res = res + pol[i] * pot	n

$$\sum_{i=0}^{n} i = \frac{n(n+1)}{2} \approx T(n) = \underline{O(n^2)}$$

Horner

n ← 0	1
for i ← 0 to n do	n+1
X ← 0	1
resultado = a[n]	1
for i ← n-1 to 0 do	n-1
resultado = resultado * X + a[i]	
print(resultado)	1

$\therefore 2n + O(n) = \Theta(n)$

$T(n) = \underline{\Theta(n)}$